CS 214 Fall 2026
Project III: A simple shell

David Menendez

Due: November 11, at 11:59 PM (ET)

For this project, you and your partner will design and implement a simple command-line shell, similar to `bash` or `zsh`. Your program, `mysh`, will provide *interactive* and *batch* modes, both of which will read and interpret a sequence of commands.

This project will provide experience of

- Posix (unbuffered) stream IO
- Reading and changing the working directory
- Spawning child processes and obtaining their exit status
- Use of `dup2()` and `pipe()`

1 Overview

Your program `mysh` reads a series of commands, each of which specifies a program (or built-in command) and a list of arguments to provide to that program (see section 2). For most commands, `mysh` will spawn one or more child processes to execute the command, wait for all child processes to terminate, and then proceed to the next command.

`mysh` itself takes up to one argument. When given one argument, it will read commands from the specified file. When given no argument, it will read commands from standard input.

When run with no argument, `mysh` will use `isatty()` to determine whether to run in *interactive mode* or *batch mode*. When run with no argument, `mysh` will always run in batch mode.

For full credit, your program must have one input loop and command parsing algorithm that works for both modes.

Your code must use `read()` to obtain input, and should be careful to (1) obtain a full command before executing it, and (2) not call `read()` after receiving a newline character until it has executed the received command. The second point is important to ensure that `mysh` is interactive and does not block waiting for additional input when it has already received a complete command.

`mysh` terminates when it receives the `exit` command, or when its input stream ends.

Interactive vs batch mode When running in interactive mode, `mysh` will print welcome and goodbye messages, and print a prompt before reading the next command. In batch mode, `mysh` will not print these messages or prompts.

When running in interactive mode, `mysh` will allow child processes to inherit its standard input stream (the terminal), except when the command specifies output redirection or a pipe. In batch mode, `mysh` will instead use `/dev/null` as the default input stream for child processes.

Welcome and good-bye When running in interactive mode, `mysh` should print a message such as "Welcome to my shell!" before printing the first command prompt. Additionally, after receiving the exit file or reaching the end of the input stream, it should print a message such as "Exiting my shell." (Feel free to choose whatever wording you like for these messages.)

Prompt format When running in interactive mode, `mysh` will print a prompt to indicate that it is ready to read input. The prompt will be the working directory, followed by `$` and a space.

However, if the working directory is within the user's home directory, the home directory portion of the path will be replaced by `~`. Thus, when the working directory is the home directory, the prompt will be "`~$` ".

You may assume that the `HOME` environment variable, obtainable using `getenv()`, contains the path of the home directory. You may also assume that the home directory will not change.

Exit status When in interactive mode, `mysh` will indicate whether a command completed successfully before printing the next prompt. If the command exited normally with a non-zero exit code, `mysh` will print a message such as "Exited with status N". If the command was terminated by a signal, `mysh` will print a message such as "Terminated by signal X", using `psignal()` or `strsignal()` to print the signal's message string.

Usage Batch mode:

```
$ cat myscript.sh
echo hello
$ ./mysh myscript.sh
hello
$
```

Batch mode with no specified file:

```
$ cat myscript.sh | ./mysh
hello
$
```

Interactive mode:

```
$ ./mysh
Welcome to my shell!
~$ cd subdir
~/subdir$ echo hello
hello
~/subdir$ cd subsubdir
~/subdir/subsubdir pwd
/home/user/subdir/subsubdir
```

```
~/subdir/subsubdir$ cd directory_that_does_not_exist
cd: No such file or directory
~/subdir/subsubdir$ cd ../..
~$ exit
mysh: exiting
$
```

2 Command format

`mysh` reads one command per line, which describes a *job*. A typical job will involve one or more child processes executing specified programs with specified arguments and possibly overridden standard input or output files.

When processing a command, `mysh` must determine the number of child processes to start and, for each one,

1. The path to the executable file (if any)
2. The list of argument strings
3. Which files to use for standard input and output (if any)

To accomplish this, `mysh` will process the command line in stages.

The command itself is made of *tokens*. For our purposes, a token is a sequence of non-whitespace characters, except that `>`, `>>`, `<`, and `|` are always a token on their own. For simplicity, you are permitted to assume that `>`, `>>`, `<`, and `|` are separated from other tokens by whitespace.

The character `#` begins a comment, which continues until the next newline character. No other symbols have meaning within a comment, and comments are not passed to commands. A command containing only a comment is treated as an empty command (that is, `mysh` does nothing and proceeds to the next command).

The token stream determines the argument list and whether the command includes redirection. We can build the argument list as we step through the token stream.

Redirection The special tokens `<`, `>`, and `>>` introduce input/output redirection (see section 2.4). The first argument following the redirection flag is not added to the argument list, but instead kept as a redirection file name.

For simplicity, you may assume that any redirection tokens occurs after normal arguments.

Pipelines The special token `|` separates two programs in a *pipeline*, each of which will have a separate argument list and file redirection targets. Additionally, `mysh` will use `pipe()` to arrange that standard output for each process except the last will be written to standard input for the next process (see section 2.5).

For simplicity, you may assume that subcommands in a pipeline will not involve redirection.

2.1 Success and failure

Each (non-empty) command will either *succeed* or *fail*.

- A command that launches a program succeeds if and only if that program exits normally with the exit code 0.
- Built-in commands succeed if and only if they receive the correct number of arguments and successfully perform their operation. (For example, `cd` would fail if `chdir()` reports an error.)
- Pipelines succeed if the last program in the pipeline succeeds.
- A command containing a syntax error, such as the sequence `< <`, will fail.

Note that `mysh` continues executing after a failed command. In the case of a syntax error, it will skip ahead to the next newline and proceed from there.

Note that `mysh` itself should always exit with status `EXIT_SUCCESS`, except when it is unable to open its argument.

2.2 Program name

The first argument determines which program is executed. This argument may be the name of a program, a path to a program file, or a "built-in" command.

Pathnames If the first argument contains a slash character, `/`, `mysh` will assume it is a path to an executable file.

Built-in commands If the first argument is the name of a built-in command described in section 3, `mysh` will execute the specified built-in. Built-ins are normally executed directly in the shell, without creating a child process.

Bare names If the first argument does not contain a slash, and is not one of the built-in commands, `mysh` will search for an executable file with the same name. For this assignment, `mysh` will search the directories `/usr/local/bin`, `/usr/bin`, and `/bin`, in that order.

You may use `access()` to determine whether an executable file with the appropriate name exists in these directories without needing to traverse the directory itself.

Note that most commands are executed in this way and are not part of the shell. Do not attempt to implement common Unix commands (cp, mv, cat, etc.) yourself!

Executable files For our purposes, we consider a file executable if it exists and we have the execute permission for it. This can be confirmed using `access()`.

2.3 Wildcards

A token containing an asterisk (`*`) is a *wildcard* (or *glob*), representing a set of files whose names match a pattern. We allow a single asterisk in a file name or in the last section of a path name. Any file in the specified directory whose name begins with the characters before the asterisk and ends with the characters after the asterisk is considered to match.

In other words, any name where we can replace a sequence of zero or more characters with an asterisk to obtain the pattern is considered a match.

Thus, `foo*bar` matches file names in the working directory that begin with "foo" and end with "bar".

Similarly, `baz/foo*bar` matches file names in the subdirectory "baz" that begin with "foo" and end with "bar".

When a command includes a wildcard token, it will be replaced in the argument list by the list of names matching the pattern.

If no names match the pattern, `mysh` should pass the token to the command unchanged.

Hidden files Patterns that begin with an asterisk, such as `*.txt`, will not match names that begin with a period.

2.4 Redirection

The tokens `<`, `>`, and `>>` are used to indicate input or output redirection. The next token is understood as a path to the redirection file. Neither token is provided to the program as an argument.

Normally, a child process will use the same standard input and output as its parent. When using file redirection, `mysh` should open the specified file in the appropriate mode and use `dup2()` in the child process to redefine file 0 or 1 before calling `execv()`.

The token `<` indicates that standard input should read from the specified file.

The token `>` indicates that standard output should write to the specified file. The file should be created if it does not exist and truncated if it does.

The token `>>` indicates that standard output should append to the specified file. The file should be opened in append mode and created if it does not exist.

Files created by `>` and `>>` should use mode 640 (`S_IRUSR|S_IWUSR|S_IRGRP`).

If `mysh` is unable to open the file in the requested mode, it should print an error message and fail the command.

2.5 Pipes

A pipe connects standard input from one program to the standard output from another, allowing data to "flow" from one program to the next. `mysh` allows for a single pipeline to contain n pipes connecting $n + 1$ processes.

Before starting the child processes, use `pipe()` to create a pipe, and then use `dup2()` to set standard output of the first process to the write end of the pipe and standard input of the second process to the read end of the pipe.

If the pipe cannot be created, `mysh` should print an error message and fail the command. Otherwise, the command succeeds if and only if the last sub-command succeeds.

2.6 Additional notes

When a wildcard does not match any files, it is included in the argument list as-is.

For simplicity, you may assume that the token immediately following a `<`, `>`, or `>>` does not contain a wildcard.

Note that more than one token may have a wildcard. Be sure to handle them in a uniform manner.

It is recommended that you use `execv()` to execute programs, as it can easily be given a variable number of arguments. Note that the arraylist code from earlier can be used to efficiently create an array by repeatedly appending items to its end, without needing to know in advance how long the array will be. This is ideal for creating an argument list in the presence of wildcard tokens.

Do not use `execvp()` or `execlp()` to start programs. They will search a list of directories different from the ones mandated for `mysh`.

A single command can redirect both standard input and standard output. These are not required to be given in any particular order. The following commands are equivalent:

```
foo bar < baz > quux
foo bar > quux < baz
```

3 Built-in commands

The commands `cd`, `pwd`, `which`, and `exit` are implemented by `mysh` itself. (Programs by these names may exist in the search directories, but you should ignore those.)

A built-in command specified by itself will usually run in the shell's process (this is necessary for `cd` to work, for example). However, a built-in command occurring in a pipeline may execute in a child process.

Although none of these commands read from standard input, and only `pwd` and `which` write to standard output, users are permitted to use redirection and pipes with built-in commands.

3.1 cd

`cd` is used to change the working directory. If no argument is given, the new directory is the home directory. If one argument is given, it specifies the new directory (this may be a relative or absolute path). In both cases, `mysh` should use `chdir()` to change the working directory.

`cd` should print an error message and fail if it is given the too many arguments, or if `chdir()` fails.

3.2 pwd

`pwd` prints the current working directory to standard output. This can be obtained using `getcwd()`.

3.3 which

`which` takes one or more arguments and prints the path to an executable for each one, mirroring the search process used to understand commands. Arguments including a slash (`/`) are printed as-is if the file is executable. For arguments not including a slash, search the three directories for an executable file by that name and print the resulting path.

Unlike the command processor, `which` does not treat the built-in commands specially. On the iLab, for example, `which which` would print `/usr/bin/which`.

`which` succeeds if it found executable files for each argument, and fails if it received no arguments or if it could not find one or more executables.

3.4 exit

`exit` ignores its arguments and does nothing. If `exit` is present in a command, `mysh` should terminate successfully once that command is complete.

While `exit` will normally appear by itself, a user could use it as a subcommand in a pipeline. For example, `exit | echo hello > foo`. Even though `exit` may be executing in a child process (where it would immediately terminate), `mysh` itself will also terminate once the remaining processes in the pipeline have terminated.

4 Submission

A system will be provided for you to declare your partnership prior to submission, details forthcoming. Determine in advance which partner will be responsible for submitting the completed assignment. Communication and coordination is key to a successful project!

Submit a Tar archive containing:

- Your source code file(s), including testing code
- Your make file
- Your README
- Any test inputs used by your testing process

Your README should be a plain text document containing the names and NetIDs of both partners.

Your test plan should be part of the README, and detail your testing strategy and test cases. Describe the scenarios you considered it important to check and how you performed those checks. Note that having a good test suite is an excellent way to detect errors that have been introduced or reintroduced into your code.